

MLOps Practical Handouts

SCSE3040 Machine Learning Operations

Bennett University | Session 2026–27

This is the master Overleaf project for Practical 01 through Practical 13. Each practical is maintained in a separate file under **practicals/**. Blank practical files are already created, so each practical can be developed one at a time without changing the project architecture.

Contents

Practical 01: Building a Reproducible ML Workbench	5
1 Why This Practical Exists	5
2 Learning Outcomes	5
3 What We Are Building	5
4 Prerequisites	6
5 Repository Structure	6
6 Pre-flight Environment Verification	6
7 Identify the Executing Python	7
7.1 Command	7
7.2 Component-wise Explanation	7
8 Inspect the Software Environment	7
9 Capture Dependency Versions	7
10 Randomness and Reproducibility	8
11 Controlled Random Generation	8
12 Generate and Persist the Course Dataset	8
12.1 Define the Destination	8
12.2 Generate the Columns	9
12.3 Construct and Write the DataFrame	9
13 SHA-256 and Dataset Identity	9
14 Build Run Provenance	10
15 Introduce Git Provenance	10
16 Student Tasks T1/T2/T3	10
17 Automated Self-check	11
18 Troubleshooting	11

19 Expected Final Filesystem	11
20 What Belongs in Git	11
21 What Must Not Be Committed	12
22 Practical Completion Checklist	12
23 Connection to P02	12
24 References	12
 Practical 02: Your First Honest Model	 14
1 Why This Practical Exists	14
2 Learning Outcomes	14
3 What We Are Building	14
4 Prerequisites	15
5 Pre-flight Verification	15
6 Establish the Shared Dataset	15
7 Load and Inspect the Dataset	16
8 Separate Features and Target	16
9 Create the Evaluation Boundary	16
9.1 Component-wise Explanation	17
10 Establish a Baseline	17
11 Fit Linear Regression	17
12 Evaluate Held-out Predictions	18
12.1 MAE	18
12.2 RMSE	18
13 Compare Model and Baseline	18
14 Inspect the Learned Coefficients	18
15 Predict One New Order	19

16 Student Task T1: Mean Versus Median Baseline	19
17 Student Task T2: A Different Split	19
18 Student Task T3: One Order In, One Number Out	20
19 Automated Self-check	20
20 Troubleshooting	20
21 Practical Completion Checklist	20
22 Connection to P03	21
23 References	21
 Practical 03: Choosing a Model Honestly	 22
1 Why This Practical Exists	22
2 Learning Outcomes	22
3 Self-contained Data Setup	22
4 Feature and Target Contract	23
5 Scientific Evaluation Architecture	23
6 Create the Outer Final Holdout	24
7 Create the Inner Diagnostic Split	24
8 Train/Validation Diagnostic Helper	25
9 Interpret the Diagnostic Gap	25
10 Fixed-model Diagnostics	26
10.1 Linear Regression	26
10.2 Unrestricted Decision Tree	26
10.3 Depth-4 Decision Tree	26
10.4 Random Forest	26
11 Build the Diagnostic Table	26
12 Five-fold Cross-validation on Development Data	27
12.1 What Five-fold CV Does	27

12.2 Why Negative MAE?	27
13 Compare Fixed Candidates by Development CV	27
14 Interpret Fold-to-fold Spread Carefully	28
15 Tune Decision Tree Depth	28
16 Select Before Looking at Final Test Results	29
17 Refit on All Development Data	29
18 Perform One Final Test	29
19 Student Task T1: Measure Overfitting Manually	29
20 Student Task T2: Tune Random Forest Depth	30
21 Student Task T3: Reusable Comparison Function	30
22 Automated Self-check	30
23 Common Execution Errors	30
24 Practical Completion Checklist	31
25 Connection to Later MLOps	31
26 References	31

Practical 01

Building a Reproducible ML Workbench

From “it ran on my laptop” to inspectable execution evidence

Course	SCSE3040 Machine Learning Operations
Institution	Bennett University
Session	2026–27
Mode	Hands-on, CPU-only

1 Why This Practical Exists

A machine-learning result is weak evidence if we cannot establish which Python executed it, which packages were installed, which random choices were made, which data were used, and which source-code revision produced the result.

This practical therefore builds the engineering substrate on which later model training, MLflow tracking, packaging, deployment, CI/CD, and monitoring will depend.

Why MLOps Cares

The deeper question is not merely “Did the code run?” It is “What exactly produced this result, and can we establish that later?”

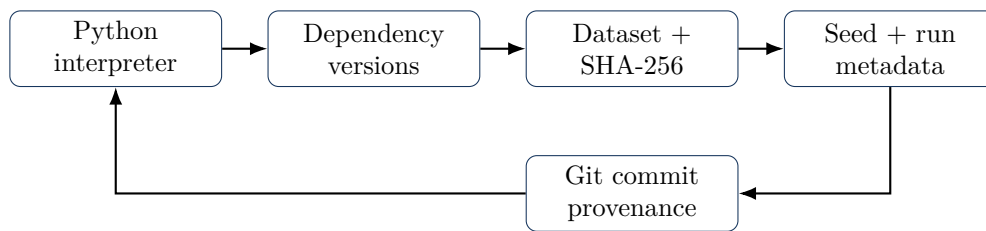
2 Learning Outcomes

By the end of the practical, a student should be able to:

- identify the Python interpreter actually executing the work;
- inspect and capture dependency versions;
- explain deterministic random-number generation and the role of a seed;
- generate the shared course dataset reproducibly;
- compute and interpret a SHA-256 dataset fingerprint;
- record run provenance in a machine-readable artifact;
- connect source-code state to a Git commit;
- distinguish files that belong in version control from generated or local-only files.

3 What We Are Building

The practical builds a small but meaningful provenance chain:



4 Prerequisites

Before starting:

- open the course repository;
- activate the project virtual environment;
- verify that Jupyter uses the intended kernel;
- work from the Practical 01 directory;
- do not create a second Git repository inside the practical folder.

5 Repository Structure

A clean course layout should conceptually resemble:

```

mlops-course/
|-- data/
|-- Practicals/
| |-- P01-reproducible-ml-workbench/
| |-- README.md
| |-- P01.ipynb
| |-- requirements.txt
| '-- .gitignore
'-- ...
  
```

6 Pre-flight Environment Verification

Execute

Run the environment-check cell before generating data or provenance.

```

import sys
from pathlib import Path

print("Python :", sys.version.split()[0])
print("Program:", sys.executable)
print("Folder :", Path.cwd())
  
```

Verify

The executable path should point to the intended course environment, and the working directory should be the practical workspace you expect.

7 Identify the Executing Python

7.1 Command

In PowerShell:

```
where.exe python
Get-Command python
python -c "import sys; print(sys.executable)"
```

7.2 Component-wise Explanation

Component	Meaning
<code>where.exe python</code>	Lists matching Python executables discoverable through the Windows command search path.
<code>Get-Command python</code>	Asks PowerShell which command it will resolve for the name <code>python</code> .
<code>python -c "..."</code>	Starts the resolved interpreter and asks Python itself to report <code>sys.executable</code> .

Stop and Interpret

The shell's answer and Python's own answer should tell a consistent story. A prompt marker such as `(.venv)` is useful, but direct verification is stronger evidence.

Why MLOps Cares

Environment identity is part of experiment provenance. Installing the right package into the wrong interpreter can make a notebook appear correct while using an unintended environment.

8 Inspect the Software Environment

Execute

Inspect the active package environment.

```
python -m pip --version
python -m pip list
```

The form `python -m pip` explicitly binds `pip` to the Python interpreter selected by `python`.

9 Capture Dependency Versions

Execute

Capture the current environment to a text artifact.

```
python -m pip freeze > requirements.txt
```


Verify

Open `requirements.txt`. It should contain package names and versions rather than being empty.

Why MLOps Cares

A dependency snapshot is not a complete reproduction of the operating system, hardware, or every transitive runtime condition. It is nevertheless a useful and inspectable part of the execution record.

10 Randomness and Reproducibility

Machine-learning workflows frequently contain random operations. Without control, repeated runs may create different data partitions, initial values, samples, or synthetic observations.

A **random seed** initializes a pseudorandom-number generator so that the same generator and algorithm can reproduce the same sequence under the same relevant conditions.

11 Controlled Random Generation

```
import numpy as np

SEED = 42
rng = np.random.default_rng(SEED)

values = rng.normal(size=5)
print(values)
```

Stop and Interpret

Restart the kernel and rerun the cell. If the generator is recreated with the same seed, the sequence should repeat. If the generator state has already advanced, the next call will produce the next values in that sequence.

12 Generate and Persist the Course Dataset

The course uses a synthetic delivery-time dataset so that every student can begin from a controlled, reproducible source.

12.1 Define the Destination

```
from pathlib import Path
import pandas as pd
import numpy as np

SEED = 42
N_ROWS = 600

DATA = Path("../") / "data" / "delivery_times.csv"
DATA.parent.mkdir(parents=True, exist_ok=True)
```

12.2 Generate the Columns

```
rng = np.random.default_rng(SEED)

distance_km = np.round(
    rng.uniform(0.5, 12.0, N_ROWS), 2
)
prep_time_min = np.round(
    rng.uniform(5, 30, N_ROWS), 0
)
traffic_level = rng.integers(1, 4, N_ROWS)
rain = rng.binomial(1, 0.25, N_ROWS)

delivery_min = np.round(
    6.0
    + 3.1 * distance_km
    + 0.65 * prep_time_min
    + 4.2 * traffic_level
    + 5.5 * rain
    + rng.normal(0, 2.5, N_ROWS),
    1,
)
```

12.3 Construct and Write the DataFrame

```
orders = pd.DataFrame({
    "distance_km": distance_km,
    "prep_time_min": prep_time_min.astype(int),
    "traffic_level": traffic_level.astype(int),
    "rain": rain.astype(int),
    "delivery_min": delivery_min,
})

orders.to_csv(DATA, index=False)

print("saved :", DATA)
print("shape :", orders.shape)
print(orders.head())
```

Verify

The file should exist at `DATA`, and the DataFrame shape should be `(600, 5)`.

13 SHA-256 and Dataset Identity

A filename is not a strong identity. Two different files can both be named `delivery_times.csv`.

```
import hashlib

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(1024 * 1024), b''):
            h.update(block)
    return h.hexdigest()
```

Why MLOps Cares

A cryptographic hash provides a compact fingerprint of file bytes. If the bytes change, the SHA-256 value is overwhelmingly likely to change as well. It establishes file identity much more strongly than a filename alone.

14 Build Run Provenance

Create a machine-readable record rather than relying on memory.

```
import json, platform, sys
from datetime import datetime, timezone

run_info = {
    "created_at_utc": datetime.now(timezone.utc).isoformat(),
    "python_version": sys.version,
    "python_executable": sys.executable,
    "platform": platform.platform(),
    "seed": SEED,
    "dataset_sha256": sha256_file(DATA),
}

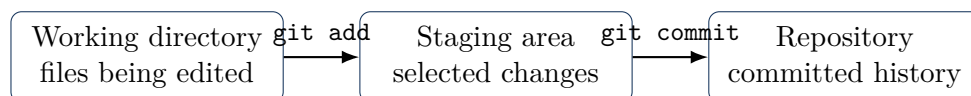
with open("run_info.json", "w", encoding="utf-8") as f:
    json.dump(run_info, f, indent=2)
```

Verify

Open `run_info.json`. Confirm that the interpreter, seed, and dataset hash are explicit and readable.

15 Introduce Git Provenance

```
git status
git add .
git commit -m "Initial reproducible workbench"
git log --oneline
```



Why MLOps Cares

A commit identifies source-code state. Together, dependency versions, dataset fingerprint, random seed, configuration, and source revision form a much stronger provenance chain.

16 Student Tasks T1/T2/T3

Complete the T1, T2, and T3 cells in the supplied `P01.ipynb`. Preserve the required variable names because the automated self-check depends on them.

For each task, follow this discipline:

1. read the requirement;
2. identify the artifact or variable that must exist;
3. implement only the required behavior;
4. inspect the result;
5. run the self-check only after the task itself makes sense to you.

17 Automated Self-check

Treat each self-check condition as a miniature software requirement, not merely as grading machinery.

A passing check should answer: *What property of my workflow has just been verified?*

18 Troubleshooting

Symptom	What to verify first
Wrong Python path	Activation and Jupyter kernel selection.
Package import fails	<code>sys.executable</code> , then <code>python -m pip -version</code> .
Different random values	Seed placement and whether generator state has already advanced.
Dataset hash differs	File bytes, generation code, seed, row order, formatting, and whether the file was edited.
Git reports many unwanted files	Review <code>.gitignore</code> before staging.
Nested repository behavior	Check that <code>git init</code> was not run inside the practical directory.

19 Expected Final Filesystem

A student-facing repository should remain clean:

```
P01-reproducible-ml-workbench/
|-- .gitignore
|-- P01.ipynb
|-- README.md
'-- requirements.txt
```

Generated experiment artifacts may exist locally during execution, but they need not all belong in the public repository.

20 What Belongs in Git

Typical candidates:

- notebook source;
- README and instructions;
- dependency specification;
- small deterministic source/configuration files;
- scripts required to reproduce the workflow.

21 What Must Not Be Committed

Do not blindly commit:

- `.venv/`;
- `__pycache__/` and bytecode;
- notebook checkpoints;
- nested `.git/` directories;
- temporary editor/OS files;
- credentials, tokens, API keys, or secrets;
- large generated artifacts unless the repository design explicitly requires them.

22 Practical Completion Checklist

- ☐ I can prove which Python executed the practical.
- ☐ I inspected and captured dependency versions.
- ☐ I can explain what a random seed does and does not guarantee.
- ☐ I generated the 600-row course dataset.
- ☐ I computed and interpreted its SHA-256 fingerprint.
- ☐ I created a machine-readable provenance record.
- ☐ I understand working directory, staging area, and committed Git history.
- ☐ T1/T2/T3 are complete.
- ☐ Automated checks pass.
- ☐ I reviewed repository hygiene before committing.

23 Connection to P02

P01 answers:

What execution context produced the data and artifacts?

P02 adds the next question:

How do we train a model and produce honest evidence on observations it did not learn from?

24 References

- Python documentation: <https://docs.python.org/3/>
- NumPy random generation: <https://numpy.org/doc/stable/reference/random/>
- Python hashlib: <https://docs.python.org/3/library/hashlib.html>

- Git documentation: <https://git-scm.com/doc>
- pip user guide: https://pip.pypa.io/en/stable/user_guide/

Practical 02

Your First Honest Model

Baseline, held-out evaluation, and the first reusable inference boundary

Course	SCSE3040 Machine Learning Operations
Institution	Bennett University
Session	2026–27
Mode	Hands-on, CPU-only

1 Why This Practical Exists

A model that performs well on the observations it learned from has not yet demonstrated that it can generalise.

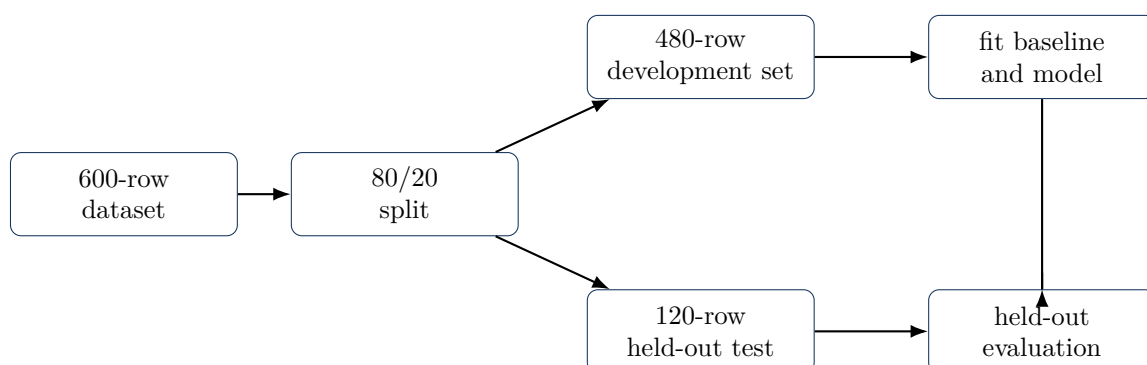
P02 introduces an evaluation boundary and a baseline so that a model score becomes interpretable evidence rather than an isolated number.

2 Learning Outcomes

Students will:

- distinguish features from the prediction target;
- create a reproducible train/test split;
- build a baseline from training information only;
- fit Linear Regression;
- evaluate held-out predictions using MAE and RMSE;
- compare the model with a baseline;
- interpret coefficients cautiously;
- retrain correctly after changing a split;
- expose one-row prediction through a small function.

3 What We Are Building



4 Prerequisites

Complete P01 first. Verify the intended Python environment and shared dataset before modelling.

5 Pre-flight Verification

```
import sys
from pathlib import Path

print("Python :", sys.version.split()[0])
print("Program:", sys.executable)
print("Folder :", Path.cwd())
```

6 Establish the Shared Dataset

P02 remains independently executable. It defines the shared dataset path and reconstructs the synthetic teaching dataset only when the file is absent.

```
from pathlib import Path
import numpy as np
import pandas as pd

DATA = Path("../") / "data" / "delivery_times.csv"
SEED = 42
N_ROWS = 600

def build_course_data(path=DATA):
    rng = np.random.default_rng(SEED)

    distance_km = np.round(
        rng.uniform(0.5, 12.0, N_ROWS), 2
    )
    prep_time_min = np.round(
        rng.uniform(5, 30, N_ROWS), 0
    )
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)

    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )

    df = pd.DataFrame({
        "distance_km": distance_km,
        "prep_time_min": prep_time_min.astype(int),
        "traffic_level": traffic_level.astype(int),
        "rain": rain.astype(int),
        "delivery_min": delivery_min,
    })
```



```
path.parent.mkdir(parents=True, exist_ok=True)
df.to_csv(path, index=False)

if not DATA.exists():
    build_course_data()

print("dataset:", DATA)
print("exists :", DATA.exists())
```

Why MLOps Cares

Automatic rebuilding is a teaching convenience for this synthetic course dataset. A production ML system should use explicit data-acquisition and validation stages rather than silently recreating missing real-world data.

7 Load and Inspect the Dataset

```
import pandas as pd

orders = pd.read_csv(DATA)
print("rows, columns:", orders.shape)
print(orders.head())
print(orders.dtypes)
```

Verify

Expected shape: (600, 5).

8 Separate Features and Target

```
FEATURES = [
    "distance_km",
    "prep_time_min",
    "traffic_level",
    "rain",
]
TARGET = "delivery_min"

X = orders[FEATURES]
y = orders[TARGET]
```

Why MLOps Cares

The ordered feature list is the beginning of an input contract. The same names and semantics must eventually survive training, packaging, serving, and monitoring.

9 Create the Evaluation Boundary

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
)
```

9.1 Component-wise Explanation

Component	Meaning
X, y	aligned feature records and targets
test_size=0.2	reserve 20 percent for held-out evaluation
random_state=42	reproduce this partition
X_train, y_train	observations available to fitting
X_test, y_test	observations withheld from fitting

Verify

Expected sizes: 480 training observations and 120 held-out observations.

10 Establish a Baseline

```
import numpy as np
from sklearn.metrics import mean_absolute_error

average_time = y_train.mean()
baseline_predictions = np.full(len(y_test), average_time)
baseline_mae = mean_absolute_error(y_test, baseline_predictions)
```

Stop and Interpret

Why `y_train.mean()` rather than `y_test.mean()`? Because the baseline predictor must obey the same information boundary as the model. Test targets are evidence, not training information.

11 Fit Linear Regression

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

Why MLOps Cares

`LinearRegression()` constructs an estimator. `fit()` changes its state by learning parameters from training observations. A fitted object is therefore a model artifact with learned state.

12 Evaluate Held-out Predictions

```
from sklearn.metrics import mean_squared_error

predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = mean_squared_error(y_test, predictions) ** 0.5

print("MAE :", mae)
print("RMSE:", rmse)
```

12.1 MAE

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

It remains in minutes, the target unit.

12.2 RMSE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Larger residuals receive more influence because they are squared before averaging.

Stop and Interpret

Do not infer from $\text{RMSE} > \text{MAE}$ alone that there are definitely a few catastrophic errors. Inspect residuals before making a stronger claim.

13 Compare Model and Baseline

```
improvement_pct = 100 * (
    baseline_mae - mae
) / baseline_mae
```

For the supplied course dataset, reference observations are approximately:

Measure	Reference
Baseline MAE	10.32 min
Model MAE	1.92 min
Model RMSE	2.48 min
Relative MAE reduction	about 81%

14 Inspect the Learned Coefficients

```
learned = pd.DataFrame({
    "feature": FEATURES,
    "coefficient": model.coef_,
})
print(learned)
print("intercept:", model.intercept_)
```

Common Failure

A fitted coefficient is a model parameter, not automatic proof of causation. Within this linear model it describes a conditional prediction relationship with the other included features held fixed.

15 Predict One New Order

```
new_order = pd.DataFrame([
    "distance_km": 5.0,
    "prep_time_min": 20,
    "traffic_level": 2,
    "rain": 0,
])

minutes = model.predict(new_order)[0]
print(minutes)
```

Why MLOps Cares

This introduces the inference boundary:

caller inputs → feature representation → model.predict → returned prediction.

Later this same boundary will sit behind an API.

16 Student Task T1: Mean Versus Median Baseline

Construct a median baseline using `y_train.median()` and store:

```
T1_median_mae
T1_which_is_better
```

The second variable must be exactly "mean" or "median".

17 Student Task T2: A Different Split

Repeat the experiment with:

```
test_size=0.30
random_state=7
```

Create:

```
X_tr2, X_te2, y_tr2, y_te2
model2
T2_mae
```

Common Failure

A new split requires a new fitting step. An already fitted model does not automatically relearn when the partition changes.

18 Student Task T3: One Order In, One Number Out

Implement:

```
predict_minutes(
    distance_km,
    prep_time_min,
    traffic_level,
    rain,
)
```

It must build a one-row DataFrame, use the original fitted model, and return one plain Python number rounded to one decimal place.

Evaluate the requested rainy order and store it in `T3_rainy_order`.

19 Automated Self-check

The supplied notebook checks nine requirements. A complete solution should report:

```
9 of 9 checks passed
```

20 Troubleshooting

Symptom	First verification
<code>ModuleNotFoundError</code>	Check notebook <code>sys.executable</code> .
<code>KeyError</code> for a feature	Inspect dataset path and column names.
Unexpected split sizes	Verify <code>test_size</code> .
Reference metrics differ substantially	Verify dataset identity and <code>random_state</code> .
T2 looks identical	Confirm <code>model2.fit(X_tr2,y_tr2)</code> actually ran.
Feature-name warning	Predict with a DataFrame using expected columns.
<code>NameError</code>	Run prerequisite cells top to bottom.

21 Practical Completion Checklist

- ☐ Dataset shape verified.
- ☐ Features and target separated.

- ☐ 480/120 primary split verified.
- ☐ Baseline uses training targets only.
- ☐ Model fitted only on training data.
- ☐ MAE and RMSE computed on held-out predictions.
- ☐ Model compared with baseline.
- ☐ Coefficients interpreted non-causally.
- ☐ T1, T2, and T3 completed.
- ☐ Self-check reports 9 of 9.

22 Connection to P03

P02 establishes that model evidence must come from unseen observations. P03 asks the harder question: how can several models and configurations be compared without repeatedly consulting the final test set?

23 References

- https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics

Practical 03

Choosing a Model Honestly

Overfitting, development-only model selection, and one final test

Course	SCSE3040 Machine Learning Operations
Institution	Bennett University
Session	2026–27
Mode	Hands-on, CPU-only

1 Why This Practical Exists

P02 separated fitting from evaluation. P03 strengthens that protocol. If a developer repeatedly looks at final test scores while choosing algorithms or hyperparameters, the test set becomes part of the human decision process even if `fit()` never receives `y_test`.

2 Learning Outcomes

Students will be able to:

- create an outer final holdout and an inner diagnostic validation split;
- compare fitting and validation errors without consuming final-test evidence;
- recognise overfitting and underfitting from observed behavior;
- perform five-fold cross-validation on development data;
- interpret mean validation MAE and fold-to-fold spread;
- tune hyperparameters without using the final test set;
- build a reusable comparison function;
- perform one final evaluation only after selection.

3 Self-contained Data Setup

```
from pathlib import Path
import numpy as np
import pandas as pd

DATA = Path("../") / "data" / "delivery_times.csv"
SEED = 42
N_ROWS = 600

def build_course_data(path=DATA):
    rng = np.random.default_rng(SEED)

    distance_km = np.round(
        rng.uniform(0.5, 12.0, N_ROWS), 2
    )
    prep_time_min = np.round(
```

```
        rng.uniform(5, 30, N_ROWS), 0
    )
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)

    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )

    df = pd.DataFrame({
        "distance_km": distance_km,
        "prep_time_min": prep_time_min.astype(int),
        "traffic_level": traffic_level.astype(int),
        "rain": rain.astype(int),
        "delivery_min": delivery_min,
    })

    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)

if not DATA.exists():
    build_course_data()

orders = pd.read_csv(DATA)
print(orders.shape)
```

Verify

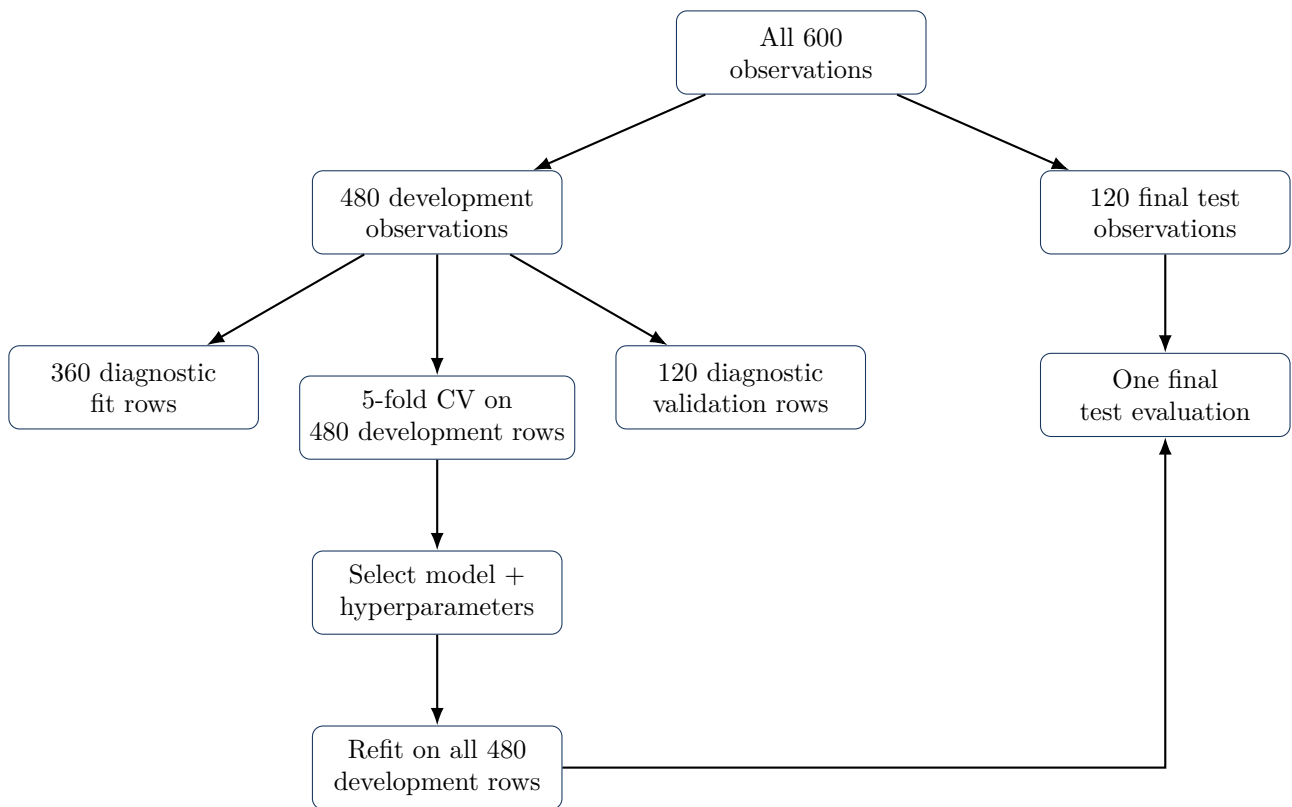
Expected dataset shape: (600, 5).

4 Feature and Target Contract

```
FEATURES = [
    "distance_km",
    "prep_time_min",
    "traffic_level",
    "rain",
]
TARGET = "delivery_min"

X = orders[FEATURES]
y = orders[TARGET]
```

5 Scientific Evaluation Architecture



Why MLOps Cares

The inner diagnostic split makes overfitting visible. Cross-validation on the 480-row development partition supports model selection. The outer 120-row test set remains untouched until the final evaluation.

6 Create the Outer Final Holdout

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
)

print("development:", X_train.shape)
print("final test :", X_test.shape)

```

Verify

Expected: 480 development observations and 120 final-test observations.

7 Create the Inner Diagnostic Split

```

X_fit, X_val, y_fit, y_val = train_test_split(
    X_train,

```

```
y_train,  
    test_size=0.25,  
    random_state=42,  
)  
  
print("diagnostic fit:", X_fit.shape)  
print("diagnostic val:", X_val.shape)
```

Verify

Expected: 360 fitting observations and 120 diagnostic validation observations.

8 Train/Validation Diagnostic Helper

```
from sklearn.metrics import mean_absolute_error  
  
def score_fit_and_validation(model, name):  
    model.fit(X_fit, y_fit)  
  
    fit_mae = mean_absolute_error(  
        y_fit,  
        model.predict(X_fit),  
    )  
    val_mae = mean_absolute_error(  
        y_val,  
        model.predict(X_val),  
    )  
  
    gap = val_mae - fit_mae  
  
    result = {  
        "model": name,  
        "fit": fit_mae,  
        "validation": val_mae,  
        "gap": gap,  
    }  
  
    print(  
        f"{name:<28} "  
        f"fit {fit_mae:5.2f} "  
        f"val {val_mae:5.2f} "  
        f"gap {gap:5.2f}"  
    )  
  
    return result
```

9 Interpret the Diagnostic Gap

$$\text{gap} = \text{validation MAE} - \text{fit MAE}$$

A large positive gap is evidence consistent with overfitting.

Stop and Interpret

A small gap is not a standalone quality score. Both fitting and validation errors can still be high. A slightly negative gap can occur if the validation subset happens to be easier.

10 Fixed-model Diagnostics

10.1 Linear Regression

```
from sklearn.linear_model import LinearRegression

linear_diag = score_fit_and_validation(
    LinearRegression(),
    "LinearRegression",
)
```

10.2 Unrestricted Decision Tree

```
from sklearn.tree import DecisionTreeRegressor

tree_full_diag = score_fit_and_validation(
    DecisionTreeRegressor(random_state=42),
    "DecisionTree (unrestricted)",
)
```

10.3 Depth-4 Decision Tree

```
tree4_diag = score_fit_and_validation(
    DecisionTreeRegressor(
        max_depth=4,
        random_state=42,
    ),
    "DecisionTree (depth 4)",
)
```

10.4 Random Forest

```
from sklearn.ensemble import RandomForestRegressor

forest_diag = score_fit_and_validation(
    RandomForestRegressor(
        n_estimators=50,
        random_state=42,
    ),
    "RandomForest (50 trees)",
)
```

11 Build the Diagnostic Table

```

diagnostic_table = pd.DataFrame([
    linear_diag,
    tree_full_diag,
    tree4_diag,
    forest_diag,
]).sort_values("validation")

diagnostic_table

```

Common Failure

Do not replace `X_val, y_val` with `X_test, y_test`. The outer test set is reserved for one final evaluation.

12 Five-fold Cross-validation on Development Data

```

from sklearn.model_selection import cross_val_score

def cross_validate_model(model, name):
    fold_mae = -cross_val_score(
        model,
        X_train,
        y_train,
        cv=5,
        scoring="neg_mean_absolute_error",
    )

    print(
        f"{name:<28} "
        f"MAE {fold_mae.mean():5.2f} "
        f"({fold_mae.std():4.2f})"
    )

    return {
        "mean": float(fold_mae.mean()),
        "std": float(fold_mae.std()),
        "folds": fold_mae,
    }

```

12.1 What Five-fold CV Does

The 480 development observations are partitioned into five folds. Each observation is used for validation once and for fitting in the other four folds.

12.2 Why Negative MAE?

scikit-learn's scoring interface follows a higher-is-better convention. Negating the returned scores restores ordinary positive MAE interpretation.

13 Compare Fixed Candidates by Development CV

```

cv_linear = cross_validate_model(
    LinearRegression(),
    "LinearRegression",
)

cv_tree4 = cross_validate_model(
    DecisionTreeRegressor(
        max_depth=4,
        random_state=42,
    ),
    "DecisionTree (depth 4)",
)

cv_forest = cross_validate_model(
    RandomForestRegressor(
        n_estimators=50,
        random_state=42,
    ),
    "RandomForest (50 trees)",
)

```

Reference development-only results are approximately:

Model	Mean MAE	Std.
Linear Regression	2.05	0.17
Decision Tree, depth 4	4.44	0.33
Random Forest, 50 trees	2.75	0.25

14 Interpret Fold-to-fold Spread Carefully

Common Failure

Fold standard deviation is descriptive variability. It is not automatically a confidence interval, and overlap of mean $\pm$ std ranges is not a formal statistical test of equivalence.

15 Tune Decision Tree Depth

```

depths = [2, 3, 4, 6, 8, None]
scores_by_depth = {}

for depth in depths:
    candidate = DecisionTreeRegressor(
        max_depth=depth,
        random_state=42,
    )

    fold_mae = -cross_val_score(
        candidate,
        X_train,
        y_train,
        cv=5,
        scoring="neg_mean_absolute_error",
    )

```

```
scores_by_depth[depth] = float(
    fold_mae.mean()
)
```

16 Select Before Looking at Final Test Results

For this synthetic dataset, Linear Regression has the lowest development-only CV MAE among the fixed candidates and is operationally simple.

```
selected_name = "LinearRegression"
selected_model = LinearRegression()
```

17 Refit on All Development Data

```
selected_model.fit(
    X_train,
    y_train,
)
```

18 Perform One Final Test

Only now use the outer final test partition.

```
final_predictions = selected_model.predict(
    X_test
)

final_test_mae = mean_absolute_error(
    y_test,
    final_predictions,
)

print("selected model :", selected_name)
print("final test MAE :", final_test_mae)
```

Why MLOps Cares

Model selection is multi-criteria. Predictive evidence matters together with latency, memory, interpretability, maintainability, stability, monitoring requirements, and deployment constraints.

19 Student Task T1: Measure Overfitting Manually

Fit a depth-12 Decision Tree on the inner fitting partition and evaluate on the inner validation partition.

Create:

```
T1_fit_mae
T1_val_mae
T1_gap
```

with:

$$T1_gap = T1_val_mae - T1_fit_mae$$

20 Student Task T2: Tune Random Forest Depth

Evaluate:

```
T2_depths = [2, 4, 6, None]
```

using five-fold CV on `X_train, y_train` only.

Create:

```
T2_scores
T2_best_depth
```

21 Student Task T3: Reusable Comparison Function

Implement:

```
compare(models)
```

It must perform five-fold CV on the development partition and return:

```
{
  "model_name": {
    "cv_mae_mean": ...,
    "cv_mae_std": ...
  }
}
```

22 Automated Self-check

The corrected checks should verify inner-split T1 calculations, development-only T2 tuning, and a development-only T3 comparison function.

A complete solution should report:

```
8 of 8 checks passed
```

23 Common Execution Errors

Mistake	Why it matters
Celebrating fit MAE	Measures fit to seen observations, not generalisation.

Using <code>X_test, y_test</code> for diagnostics	Consumes final evidence before final evaluation.
CV on full <code>X, y</code>	Leaks outer-test observations into model selection.
Forgetting the minus sign	Misreads scikit-learn's scoring convention.
Changing several factors together	Makes interpretation difficult.
Printing but not returning from <code>compare()</code>	Prevents downstream automation from consuming metrics.

24 Practical Completion Checklist

- ☐ Dataset shape verified.
- ☐ Outer 480/120 split verified.
- ☐ Inner 360/120 diagnostic split verified.
- ☐ Diagnostics use only fit/validation data.
- ☐ Five-fold CV uses only development data.
- ☐ Hyperparameter tuning does not use final-test data.
- ☐ Model choice is recorded before final-test evaluation.
- ☐ Selected model is refitted on all development data.
- ☐ Final test is evaluated once.
- ☐ T1/T2/T3 are complete.
- ☐ Self-check passes.

25 Connection to Later MLOps

P03 concept	Later manifestation
Repeated model comparison	MLflow experiment runs
Hyperparameters	tracked parameters
MAE and fold statistics	tracked metrics
Reusable comparison function	modules and automated pipelines
Self-checks	pytest and CI
Selection evidence	governance and promotion decisions
Untouched final evaluation	release gates and production validation
Trade-off reasoning	deployment architecture decisions

26 References

- https://scikit-learn.org/stable/modules/cross_validation.html
- <https://scikit-learn.org/stable/modules/tree.html>
- <https://scikit-learn.org/stable/modules/ensemble.html>
- https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html